

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 4 – ENKAPSULASI



Ardian Nafis Samudra

103132400012

PS1SD-05-reg-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

- **Enkapsulasi & Access Modifier:** Enkapsulasi adalah teknik membungkus data (atribut) dan metode ke dalam satu class untuk melindungi data dari akses langsung (data hiding). Hal ini diatur menggunakan modifier private (hanya bisa diakses di dalam class) dan public (dapat diakses dari luar class).
- **Getter dan Setter:** Method perantara yang digunakan untuk membaca (get) dan mengubah (set) atribut ber-modifier private. Setter juga berfungsi menerapkan validasi logika sebelum nilai disimpan.
- **Overloading Constructor:** Teknik mendefinisikan beberapa constructor dengan nama sama tetapi beda parameter dalam satu class, memberikan fleksibilitas saat pembuatan objek.
- **Anggota Statis & Konstanta:** Atribut dan method bernilai static terikat pada class (bukan objek individual) dan dapat dipanggil langsung tanpa instansiasi. Kombinasi public static final digunakan untuk membuat nilai konstanta yang bernilai tetap.
- **Package dan Import:** Package digunakan untuk mengelompokkan class berdasarkan fungsinya ke dalam struktur folder terpisah. Kata kunci import dipakai untuk mengakses class yang berada di package lain.

B. Guided

1. Manusia.java & DemoManusia.java

```
package guidedmngg04;

public class Manusia {
    // Definisi atribut
    private String nama;
    private int umur;

    //4.2 constructor
    public Manusia(){};
    public Manusia(String nama){
        this.nama = nama;
    }
    public Manusia(String nama, int umur){
        this.nama = nama;
        this.umur = umur;
    }

    //method setter
    public void setNama(String a){
        nama = a;
    }

    public void setUmur(int umur){
        this.umur = umur;
    }
}
```

```

    }

    //method getter
    public String getNama(){
        return nama;
    }

    public int getUmur(){
        return umur;
    }
}

package guidedmagg04;

public class DemoManusia {
    public static void main(String[] args) { // program utama
        Manusia arrMns[] = new Manusia[3]; // buat array of Object
        Manusia objMns1 = new Manusia(); // constructor pertama
        objMns1.setNama("Gatot");
        objMns1.setUmur(76);
        //constructor kedua

        Manusia objMns2 = new Manusia("Rangga");

        //constructor ketiga
        Manusia objMns3 = new Manusia("Joko", 13);
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;
        for (int i = 0; i < 3; i++) {

            System.out.println("Nama : " + arrMns[i].getNama());
            System.out.println("Umur : " + arrMns[i].getUmur());
            System.out.println();
        }
    }
}

```

Output :

```

Nama : Gatot
Umur : 76

```

```

Nama : Rangga
Umur : 0

```

```

Nama : Joko
Umur : 13

```

```

-----
BUILD SUCCESS
-----

```

```

Total time: 2.418 s
Finished at: 2026-10-07T18:40:22+07:00

```

2. HargaPulsa.java, HargaToken.java & Main.java

```
package guidedmgg04;

public class HargaPulsa {
    public void info() {
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}

package guidedmgg04;

public class HargaToken {
    public void info() {
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
        System.out.println("Rp. 500000");
    }
}

package guidedmgg04;

public class Main {

    public static void main(String[] args) {
        HargaToken objectToken = new HargaToken();
        objectToken.info();
        HargaPulsa objectPulsa = new HargaPulsa();
        objectPulsa.info();
    }
}
```

Output :

```
Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000
-----
BUILD SUCCESS
-----
Total time: 4.136 s
Finished at: 2026-10-07T18:43:51+07:00
-----
```

C. Unguided

1. Soal Unguided 1

Kamu sedang membangun program kecil untuk mencatat dan memeriksa kualitas dataset sebelum dianalisis, seperti tahap awal pengecekan *missing value*. Setiap dataset punya nama, jumlah baris, jumlah kolom, dan jumlah sel yang kosong (*missing*).

Petunjuk Umum :

Buat 3 package dengan struktur berikut:

Source Packages

```
├── model
│   └── Dataset.java
├── laporan
│   └── LaporanDataset.java
└── main
    └── Main.java
```

1. Class “Dataset” (package “model”)

Atribut (implementasikan enkapsulasi):

Atribut	Tipe	Keterangan
`nama`	`String`	Nama dataset
`jumlahBaris`	`int`	Banyak baris (observasi)
`jumlahKolom`	`int`	Banyak kolom (fitur)
`jumlahMissing`	`int`	Banyak sel yang kosong

Ketentuan tambahan:

- Buat konstanta `BATAS\_MISSING` bernilai `5.0` (persen). Dataset dengan missing diatas batas ini dianggap perlu dibersihkan.
- Buat variabel milik class yang menghitung total objek “Dataset” yang telah dibuat, beserta method untuk ambil nilainya tanpa perlu membuat objek.

Constructor (ada 3, overloading):

1. Tanpa parameter
2. Hanya parameter `nama`
3. Lengkap: `nama`, `jumlahBaris`, `jumlahKolom`, `jumlahMissing`

Setiap constructor harus menambah hitungan total dataset.

Method:

- Getter dan setter untuk semua atribut.
- Pada setter `jumlahBaris`, `jumlahKolom`, dan `jumlahMissing`, buat agar nilai tidak berubah.
- `getPersentaseMissing()` mengembalikan `double`: persentase sel kosong dibanding total sel (`baris × kolom`). Jika total sel = 0, kembalikan `0`.
- `perluDibersihkan()` mengembalikan `boolean`: `true` jika persentase missing melebihi `BATAS\_MISSING`.

2. Class “LaporanDataset” (package “laporan”)

Buat method `cetak` yang menerima satu objek `Dataset` dan menampilkan laporannya dengan format:

```
=== Laporan Dataset ===
Nama      : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing    : 866 sel (8.10%)
Status     : Perlu dibersihkan
```

Status berisi `Perlu dibersihkan` atau `Bersih` sesuai hasil dari `perluDibersihkan()`.

3. Class “Main” (package “main”)

Buat tiga objek, masing-masing dengan constructor berbeda:

Objek	Cara pembuatan	Data
1	Constructor tanpa parameter, lalu isi memakai setter	Titanic, 891 baris, 12 kolom, 866 missing
2	Constructor 1 parameter	Wine Quality
3	Constructor lengkap	Iris, 150 baris, 5 kolom, 0 missing

Simpan ketiganya dalam array, cetak laporannya dengan perulangan, lalu tampilkan di akhir:

Total dataset dibuat : 3

Jawaban :

- Package “model”, Class “Dataset.java”

```
package model;

public class Dataset {
    public static final double BATAS_MISSING = 5.0;
    private static int totalDataset = 0;

    private String nama;
    private int jumlahBaris;
    private int jumlahKolom;
    private int jumlahMissing;

    // Constructor 1: Tanpa parameter
    public Dataset() {
        totalDataset++;
    }

    // Constructor 2: Parameter nama
    public Dataset(String nama) {
        this.nama = nama;
        totalDataset++;
    }

    // Constructor 3: Parameter lengkap
    public Dataset(String nama, int jumlahBaris, int jumlahKolom, int
jumlahMissing) {
        this.nama = nama;
        setJumlahBaris(jumlahBaris);
        setJumlahKolom(jumlahKolom);
        setJumlahMissing(jumlahMissing);
        totalDataset++;
    }

    public static int getTotalDataset() {
        return totalDataset;
    }

    public String getNama() {
        return nama;
    }

    public void setNama(String nama) {
        this.nama = nama;
    }
}
```

```

    }

    public int getJumlahBaris() {
        return jumlahBaris;
    }

    public void setJumlahBaris(int jumlahBaris) {
        if (jumlahBaris >= 0) {
            this.jumlahBaris = jumlahBaris;
        }
    }

    public int getJumlahKolom() {
        return jumlahKolom;
    }

    public void setJumlahKolom(int jumlahKolom) {
        if (jumlahKolom >= 0) {
            this.jumlahKolom = jumlahKolom;
        }
    }

    public int getJumlahMissing() {
        return jumlahMissing;
    }

    public void setJumlahMissing(int jumlahMissing) {
        if (jumlahMissing >= 0) {
            this.jumlahMissing = jumlahMissing;
        }
    }

    public double getPersentaseMissing() {
        int totalSel = jumlahBaris * jumlahKolom;
        if (totalSel == 0) {
            return 0.0;
        }
        return ((double) jumlahMissing / totalSel) * 100.0;
    }

    public boolean perluDibersihkan() {
        return getPersentaseMissing() > BATAS_MISSING;
    }
}

```

- Package “laporan”, Class “LaporanDataset.java”

```
package laporan;

import model.Dataset;

public class LaporanDataset {

    public void cetak(Dataset dataset) {
        System.out.println("=== Laporan Dataset ===");
        System.out.println("Nama      : " + dataset.getNama());
        System.out.println("Jumlah Baris : " + dataset.getJumlahBaris());
        System.out.println("Jumlah Kolom : " + dataset.getJumlahKolom());
        System.out.printf("Missing      : %d sel (%.2f%%)\n",
            dataset.getJumlahMissing(), dataset.getPersentaseMissing());

        String status = dataset.perluDibersihkan() ? "Perlu dibersihkan" :
        "Bersih";
        System.out.println("Status      : " + status);
        System.out.println();
    }
}
```

- Package “main”, Class “Main.java”

```
package main;

import model.Dataset;
import laporan.LaporanDataset;

public class Main {
    public static void main(String[] args) {
        // Objek 1
        Dataset d1 = new Dataset();
        d1.setNama("Titanic");
        d1.setJumlahBaris(891);
        d1.setJumlahKolom(12);
        d1.setJumlahMissing(866);

        // Objek 2
        Dataset d2 = new Dataset("Wine Quality");

        // Objek 3
        Dataset d3 = new Dataset("Iris", 150, 5, 0);
    }
}
```

```

// Array Objek
Dataset[] daftarDataset = { d1, d2, d3 };

// Cetak Laporan
LaporanDataset laporan = new LaporanDataset();
for (Dataset ds : daftarDataset) {
    laporan.cetak(ds);
}

System.out.println("Total dataset dibuat : " + Dataset.getTotalDataset());
}
}

```

Output :

```

=== Laporan Dataset ===
Nama      : Titanic
Jumlah Baris : 891
Jumlah Kolom : 12
Missing    : 866 sel (8,10%)
Status     : Perlu dibersihkan

```

```

=== Laporan Dataset ===
Nama      : Wine Quality
Jumlah Baris : 0
Jumlah Kolom : 0
Missing    : 0 sel (0,00%)
Status     : Bersih

```

```

=== Laporan Dataset ===
Nama      : Iris
Jumlah Baris : 150
Jumlah Kolom : 5
Missing    : 0 sel (0,00%)
Status     : Bersih

```

```

Total dataset dibuat : 3

```

```

-----
BUILD SUCCESS
-----

```

```

Total time: 4.096 s
Finished at: 2026-10-07T19:05:09+07:00
-----

```

Penjelasan :

Output program menampilkan hasil pemeriksaan kualitas dari tiga objek dataset yang disimpan dalam array dan dicetak menggunakan perulangan. Dataset "Titanic" memiliki 891 baris, 12 kolom (total 10.692 sel), dan 866 sel kosong, sehingga menghasilkan persentase missing value sebesar 8,10%. Karena nilai ini melebihi BATAS_MISSING (5,0%), method perluDibersihkan() mengembalikan nilai true dan menetapkan statusnya menjadi "Perlu dibersihkan". Sebaliknya, dataset "Wine Quality" (yang baru diinisialisasi nama dataset-nya) serta "Iris" memiliki persentase missing value sebesar 0,00%, sehingga status keduanya dicetak sebagai "Bersih". Di akhir output, program memanggil method static getTotalDataset() yang berhasil

menampilkan angka 3, membuktikan bahwa pembuatan ketiga objek dari berbagai constructor telah dihitung secara kumulatif oleh variabel milik class.

D. Kesimpulan

- **Penerapan Enkapsulasi:** Penggunaan access modifier private pada atribut yang diakses melalui method getter dan setter berhasil melindungi integritas data serta memungkinkan penerapan validasi logika (seperti mencegah input nilai negatif).
- **Fleksibilitas Inisialisasi:** Constructor overloading memberikan fleksibilitas dalam membuat objek Dataset sesuai dengan jumlah parameter data awal yang tersedia.
- **Penggunaan Anggota Statis & Konstanta:** Variabel dan method bernilai static mempermudah pelacakan data tingkat class (seperti menghitung total objek yang dibuat), sedangkan kombinasi public static final efektif digunakan untuk menetapkan konstanta acuan tetap (BATAS_MISSING).
- **Modularitas Kode:** Pengelompokan class ke dalam package terpisah (model, laporan, dan main) membuat struktur proyek Java menjadi lebih rapi, terorganisir, dan mudah dipelihara.